TP : Builds and Five-Environment Promotion Pipeline

Student Name : _____

EPITA : École d'ingénieurs en informatique

jakub.motysia@epita.fr

sultan.amiour@epita.fr

The DevOps Approach

Antoine GHORRA

06 November, 2026

LAB 04 | Pre-class installation and launch guide

Instructor-added supplement to the students' original "Builds and Five-Environment Promotion Pipeline". Read this BEFORE Mission 0. The original 12-page student-authored lab remains unchanged in the combined PDF.

Status: the GitHub starter repository referenced by the student handout has NOT been verified or downloaded here. Its actual contents and availability must be checked before distributing the lab. This supplement does not replace the student-authored source code.

A. Required environment and downloads

Component	Requirement	Where / why
Operating system	Windows 11 + WSL2 Ubuntu; macOS; or Ubuntu Linux	All command examples use a Bash-like terminal.
Docker	Running Docker Desktop/Engine with Linux containers	https://docs.docker.com/get-docker/
Git	Git CLI, default branch main	https://git-scm.com/downloads
Node.js / npm	Node.js 20+ locally; npm bundled	https://nodejs.org/en/download ; Docker exercises intentionally use Node 16 and Node 20 images.
Shell utilities	bash, curl, du, time, chmod	WSL2/macOS/Linux terminal; Windows cmd/PowerShell alone is not the lab shell.
GitHub	Account and permission to create a repository (Path A only)	GitHub Actions requires a repository and an Actions secret.
Internet	Needed before class	Clone external starter and pull Docker images; npm registry access may be required.

B. First-time OS installation (do before class)

Windows 11

- Install WSL2 and Ubuntu using Microsoft's official procedure: https://learn.microsoft.com/windows/wsl/install. Reboot if requested.
- Install Docker Desktop and enable Settings > Resources > WSL Integration for your Ubuntu distribution; use the Linux container backend. Start Docker Desktop.
- Install Git and Node.js inside Ubuntu/WSL, or use the official Node.js distribution installed in that Linux environment. Run all lab commands from the Ubuntu shell, preferably under your Linux home directory (~/lab).

macOS

- Install Docker Desktop from the official Docker site; open it and wait for its engine to start.
- Install Git and Node.js 20+ through official downloads or Homebrew (brew install git node). Use Terminal and verify curl, bash, du and time are present.

Ubuntu Linux

- Install Docker Engine following Docker's official Ubuntu guide: https://docs.docker.com/engine/install/ubuntu/. Start the service and verify that your user can access it.
- Install Git, curl, Bash and standard utilities: sudo apt update && sudo apt install -y git curl bash coreutils time.

- Install Node.js 20+ using an official Node.js distribution or an LTS version manager. Avoid relying on an old distro-provided Node.js release.

C. Run preflight (copy/paste)

```
git --version
node --version
npm --version
docker version
docker info
docker run --rm hello-world
curl --version
```

The accompanying preflight shell script checks these programs and Docker daemon access. If a command fails, fix that prerequisite BEFORE moving to the timed exercise.

D. Correct launch sequence from a clean Bash shell

Use the following normalized commands, because text extraction and visual spacing in the original PDF can make long commands difficult to copy. These commands follow the intended Mission 0 workflow.

```
mkdir -p ~/lab/dev-machine ~/lab/qa-machine
git init --bare -b main ~/lab/remote.git
docker pull node:16
docker pull node:20
docker pull node:20-alpine
git clone -o ogn https://github.com/devopsapproach/04-citybike-sdlc ~/lab/dev-machine/citybike
cd ~/lab/dev-machine/citybike
git remote set-url ogn ~/lab/remote.git
ls -la
```

IMPORTANT: If git clone returns “Repository not found”, prompts for unexpected credentials, or lacks the files below, stop. Ask the student authors for an accessible copy of their original repository. Do not improvise replacements for missing assessment scripts.

E. Verify all source files BEFORE running the lab

```
test -f server.js && echo "OK: server.js"
test -f package.json && echo "OK: package.json"
test -f package-lock.json && echo "OK: package-lock.json"
test -d public && echo "OK: public/"
for gate in staging uat oat preprod prod; do
  test -f "scripts/test-${gate}.sh" || echo "MISSING: scripts/test-${gate}.sh"
done
npm ci
npm run build
npm start
```

In a SECOND terminal, run:

```
curl -fsS http://localhost:4173/health
curl -fsS http://localhost:4173/api/stations
```

Expect /health to indicate an operational development environment and the build manifest to exist. Stop the npm server with Ctrl+C before running Docker on host port 4173. Use npm ci when package-lock.json is supplied; npm install is appropriate only if the student authors intentionally provide no lockfile and authorize generating one.

F. How to run the two-terminal simulation safely

- Terminal A = Developer; Terminal B = QA / Project Lead. Both can run as separate shell windows on one machine; “two machines” are simulated by separate Docker containers and worktrees.
- Use Docker node:16 strictly to reproduce the incompatible-runtime error; Node 16 is obsolete and must not be treated as a secure production recommendation.
- Before each docker run -p 4173:4173, stop the previous process/container. The five temporary gate containers may use host port 4174 to keep “production” on 4173 untouched.
- For Mission 4, run each test script only after the required container is healthy. Confirm that a failed gate stops promotion; do not claim a successful hosted GitHub Actions run without evidence.

G. GitHub Actions path: access and secret setup

- Path A requires a newly created personal GitHub repository, Actions enabled and a training-only secret named PROD_API_KEY in Settings > Secrets and variables > Actions. Never paste a real production key into a PDF, terminal screenshot or repository.
- If GitHub authentication, internet or runner quota blocks Path A, choose the local shell pipeline (Path B) as permitted by the original document. Only use the provided gate scripts after verifying their presence.
- Before pushing, verify git remote -v. Change the remote from the local bare repository only when following the original instructions for the hosted path. Confirm the main branch and tag v1 are pushed.

H. Quick troubleshooting

Symptom	Do this first
docker: cannot connect to daemon	Start Docker Desktop/Engine; check docker info and WSL integration.
clone fails / files missing	Ask the student authors for access to the exact original repo and five gate scripts.
npm ci fails	Check package-lock.json; inspect npm/node versions and registry access.
port 4173 already allocated	Stop npm start or remove the previous test container.
node:16 crashes with structuredClone	Expected demonstration; record the error, then test Node 20.
permission denied on scripts	Run bash scripts/test-staging.sh, or chmod +x scripts/test-*.sh on Unix.
GitHub action fails immediately	Check YAML indentation, trigger, repository permission, workflow logs and secret presence.
Docker image larger than anticipated	Measure actual size; cache state, architecture and image versions vary.

I. Instructor sign-off checklist

☐ The exact original GitHub starter repo is accessible to every intended participant.

☐ All required application files and five test gate scripts are present.

☐ A fresh-machine npm ci + npm run build succeeds.

☐ /health and /api/stations respond on Node 20.

☐ The deliberate Node 16 crash reproduces on the chosen architecture.

☐ The multi-stage Dockerfile can be built, and v1 gate scripts pass.

☐ The local or hosted pipeline can block defective v2.

☐ The rollback restores the known-good version.

Scenario

Your team ships a small Node.js CityBike app. Terminal A (the Dev) pushes a new version straight to production because it "works on my machine". Terminal B (QA Engineer & Project Lead) detects the outage, decides to roll back, and applies the rollback on every environment. The roles are simulated in this solo lab.

Learning outcomes

- Explain the SDLC and where staging, UAT, OAT, preprod and prod fit.
- Reproduce a "works on my machine" crash on a clean environment.
- Dockerize an app with a single-stage `Dockerfile`, then a multistage one, and compare image sizes.
- Promote one image through every stage using environment variables.

Terminal Roles

Role	Who	Owns
Terminal A	Dev	App code, Dockerfiles, the v2 "feature"
Terminal B	QA Engineer	Clean-machine checks, promotion, rollback

3.3h in Action

Time	Mission
00:00 – 00:15	0 Setup & baseline
00:15 – 00:35	1 SDLC & Crash
00:35 – 01:05	2 Naïve single-stage dockerfile
01:05 – 01:45	3 Fix cache, multi-stage
01:45 – 02:35	4 Five env gates
02:35 – 03:10	5 Bad push & Decision
03:10 – 03:20	6 Deliverables

Mission 0 – Setup & baseline (15min)

Both terminals: create the folders and the bare repo that plays the role of GitHub (the SCM step).

```
>>> mkdir -p ~/lab/remote.git ~/lab/dev-machine ~/lab/qa-machine
>>> cd ~/lab/remote.git && git init --bare
>>> docker pull node:16
>>> docker pull node:20
>>> docker pull node:20-alpine
```

Terminal A: load the CityBike starter (repo `04-citybike-sdlc`) into ~/lab/dev-machine/citybike, point it at the bare repo, then verify functionality on your machine.

```
>>> git clone -o ogn https://github.com/devopsapproach/04-citybike-sdlc ~/lab/dev-
... machine/citybike
>>> cd ~/lab/dev-machine/citybike
# Redirect the remote from GitHub to the local bare repository
>>> git remote set-url ogn ~/lab/remote.git
>>> npm install
>>> npm run build
>>> npm start
# second tab: curl localhost:4173/health, then Ctrl+C in the first one
>>> du -sh node_modules
```

Expected: `npm run build` prints the build manifest and creates `dist/build-manifest.json`, and `/health` answers with `ok: true, environment: development, hasBuildManifest: true` and `secureKeyConfigured: false`. Write down the size of `node_modules` and the ten `devDependencies` listed in `package.json`, you will need them in Mission 2.

Now Terminal A plays the Dev. On your Node 20 machine you use `structuredClone()` to keep a snapshot of the stations. In `server.js`, add this line right after the `stations` array and serve it from `/api/stations`:

```
const stationsSnapshot = structuredClone(stations);
// in the /api/stations route
return send(res, 200, { stations: stationsSnapshot });
```

```
>>> npm start
```

Verify the route functions by curling localhost:4173/api/stations, CTRL C after that.

```
>>> git add . && git commit -m 'v1: station snapshot'
>>> git push -u ogn main
```

Expected: the API still answers with the four stations. It works on your machine, ship it.

Mission 1 – SDLC & crash (20min)

SDLC = Software Development Lifecycle: every stage from a ticket (a card with the acceptance criteria of a user story) to a running production server.

Ticket → SCM (GitHub) → Build → Dev → Staging → UAT → OAT → Preprod → Prod

Stage	Question it answers
Staging	Does it integrate in an environment close to prod?
UAT	Do the users / the business accept it against the ticket?
OAT	Can ops deploy, monitor and roll it back?
Preprod	Does it work with exactly the prod config?
Prod	Does it serve real users?

Terminal B: clone the repo and run server.js on two "different machines", clean containers with nothing installed.

```
>>> cd ~/lab/qa-machine
>>> git clone ~/lab/remote.git citybike && cd citybike
>>> docker run --rm -it -v $PWD:/app -w /app -p 4173:4173 node:16 node server.js
>>> docker run --rm -it -v $PWD:/app -w /app -p 4173:4173 node:20 node server.js
```

Expected: on node:16 the server crashes at startup with a ReferenceError about structuredClone; on node:20 it prints that CityBike is listening. Stop it with Ctrl+C. Copy the exact error into your report.

Questions: A) What does the crash prove? B) Name three other differences between two machines that could cause the same kind of failure. C) Which SDLC stage should catch this, and how?

Mission 2 – Naïve single-stage dockerfile (30min)

Terminal A: write the naive Dockerfile on purpose, with exactly these instructions and no .dockerignore yet.

```
# TODO: FROM node:20
# TODO: COPY . .
# TODO: RUN npm ci
# TODO: RUN npm run build
# TODO: expose port 4173 and start with node server.js
```

Measure it. Then change one line in `public/` (a comment is enough) and build again.

```
>>> time docker build -t citybike:naive .
>>> docker images citybike
# edit one line in public/, then:
>>> time docker build -t citybike:naive .
>>> docker run --rm -p 4173:4173 citybike:naive
```

Expected: the app answers, the image is huge (around a gigabyte or more), and the second build reinstalls all ten `devDependencies` even though you only touched one front-end line. Record your own numbers.
Deliverable: a table with the first build time, the rebuild time after the one-line change and the image size.
Questions: A) Why did a one-line change in `public/` run `npm ci` again? B) Where does the size come from? Name the three biggest contributors. C) Which files from your machine ended up inside the image that should not be there?

Mission 3 – Fix cache, multi-stage (40min)

Terminal A, step 1: fix the rebuild latency. Reorder the `Dockerfile` so that dependencies are installed before the source is copied, and write the `.dockerignore`.

```
# TODO: copy only package.json and package-lock.json
# TODO: install the dependencies
# TODO: copy the rest of the source
# TODO: build, expose and start as before


# TODO: .dockerignore, list what must NOT be copied into the image
```

Rebuild, change one line in `public/` again, rebuild again and compare the two times with Mission 2.
Terminal A, step 2: try the tempting shortcut. Change `RUN npm ci` to `RUN npm ci --omit=dev` in the single-stage `Dockerfile` and build again.

```
>>> docker build -t citybike:shortcut
```

Expected: the build fails with `BUILD ERROR: Necessary devDependencies missing`. Write down why: what does `npm run build` need, and when is it needed?
Terminal A, step 3: fix the image bloat with a real multi-stage build. The first stage builds, the final stage only receives what the app needs to run.

```
# TODO: stage build: FROM node:20, accept the build arg GITHUB_SHA (default local-
dev) as an ENV
# TODO: copy the package files, npm ci, copy the source, npm run build
```

```
# TODO: stage run: FROM node:20-alpine
# TODO: copy only server.js, public/ and dist/build-manifest.json from the build stage
# TODO: run as the non-root user node, expose port 4173, start with node server.js

>>> docker build -t citybike:v1 .
>>> docker images citybike
>>> docker run --rm citybike:v1 ls node_modules
>>> docker run -d --name citybike-test -p 4173:4173 citybike:v1
>>> curl localhost:4173/health
>>> docker rm -f citybike-test
```

Expected: the image is a fraction of the naive one, ls `node_modules` fails because none of the ten `devDependencies` (TypeScript included) are in the final image, and `/health` still shows `hasBuildManifest: true`. `citybike:v1` is now your known good version. Commit, push and tag it.

```
>>> git add . && git commit -m 'Dockerfile: cache + multi-stage' && git push
>>> git tag v1 && git push ogn v1
```

Deliverable: a table with three rows (naive, cache fixed, multi-stage): image size and rebuild time after a one-line change.
Questions: A) Why does the order of `COPY` and `RUN npm ci` matter for the cache? B) Why does the final stage need only those three things, and why is `hasBuildManifest` still true? C) What do you gain, for security and for ops, by leaving the `devDependencies` out?

Mission 4 – Five environmental gates (50min)

Terminal B: the same immutable image moves sequentially through five gates. The five gate test scripts (`scripts/test-*.sh`) are already written and must **not** be modified. They are passive `HTTP` and image-inspection suites that expect your pipeline to start the container on `TARGET_URL` (`http://localhost:4173`), wait until `/health` responds, execute the gate script, and clean up the container. Build once, promote the same image. If any gate exits non-zero, the pipeline stops immediately and nothing after it runs.

Script	Gate	Required Container Config	What the script proves
scripts/test-staging.sh	Staging	-e APP_ENV=staging	/health returns ok: true with hasBuildManifest: true, and /api/stations returns 200 with station data.
scripts/test-uat.sh	UAT	-e APP_ENV=uat	UI assets (/, /app.js, /styles.css) return 200; /api/stations returns 4 stations with Riverside Park at 0 bikes; valid reservation for central returns 201; invalid email and empty riverside station return 422.
scripts/test-oat.sh	OAT	-e APP_ENV=oat	Malformed JSON returns 400; >10KB payload flood is rejected (413/000); server survives a 50-request concurrent burst.
scripts/test-preprod.sh	Preprod	-e APP_ENV=preprod -e PROD_API_KEY=<key>	Image size is <220MB; node_modules/typescript is absent from the runtime image; /health reports environment: "preprod" and secureKeyConfigured: true.
scripts/test-prod.sh	Prod	-e APP_ENV=production -e PROD_API_KEY=<key>	/health reports environment: "production" and secureKeyConfigured: true (fails if using the fallback key).

For every gate in your pipeline, your orchestrator (GitHub Actions or `scripts/pipeline.sh`) must execute this exact lifecycle:

1. Remove any leftover container on port `4173` (`docker rm -f citybike-gate 2>/dev/null || true`).
2. Start the image detached (`docker run -d --name citybike-gate -p 4173:4173 ...`) with the **Required Container Config** from the table above.
3. Poll `http://localhost:4173/health` in a retry loop until it answers (use a real retry loop with a timeout, not a fixed `sleep`).
4. Run the gate's test script (`./scripts/test-<gate>.sh`). Note: test-preprod.sh also requires `IMAGE_NAME=<image>` exported in the shell environment so it can inspect the image size.

5. Stop and remove the container (`docker rm -f citybike-gate`) before the next gate starts—except in `Prod` on local runs if you want the production container (`citybike-prod`) to remain running.

P.S. pipeline.sh should run citybike-gate (gates 1–4) on -p 4174:4173 with TARGET_URL=http://localhost:4174, reserving host port 4173 (-p 4173:4173) exclusively for citybike-prod in the Prod gate.

```
# Point 'ogn' to your new private GitHub repo and push main + tag v1:
>>> git remote set-url ogn https://github.com/<your-username>/<your-repo>.git
>>> git push -u ogn main
>>> git push ogn v1
```

Path A (GitHub Actions): Create a private GitHub repository, point your `ogn` remote to it, configure a repository secret named `PROD_API_KEY` (Settings → Secrets and variables → Actions), and create `.github/workflows/sdlc.yml` from scratch. The image is built once in the `build` job, saved as a `.tar` artifact, and loaded by every downstream job:

Structure `.github/workflows/sdlc.yml` as follows:

```
name: sdlc
on:
pull_request:
  branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    # TODO:
    # 1. uses: actions/checkout@v4
    # 2. docker build --build-arg GITHUB_SHA=${{ github.sha }} -t citybike:${{
github.sha }} .
    # 3. docker save citybike:${{ github.sha }} -o /tmp/citybike.tar
    # 4. uses: actions/upload-artifact@v4 (upload /tmp/citybike.tar as 'citybike-
image')

  staging:
    needs: build
    runs-on: ubuntu-latest
```

```
    # TODO:
    # 1. uses: actions/checkout@v4
    # 2. uses: actions/download-artifact@v4 (download 'citybike-image' to /tmp)
    # 3. docker load -i /tmp/citybike.tar
    # 4. Start container (APP_ENV=staging), poll /health until ready,
    #    run ./scripts/test-staging.sh, and clean up container

  uat:
    needs: staging
    runs-on: ubuntu-latest
    # TODO: same pattern with APP_ENV=uat and ./scripts/test-uat.sh

  oat:
    needs: uat
    runs-on: ubuntu-latest
    # TODO: same pattern with APP_ENV=oat and ./scripts/test-oat.sh

  preprod:
    needs: oat
    runs-on: ubuntu-latest
    # TODO: same pattern with APP_ENV=preprod, PROD_API_KEY=${{
  secrets.PROD_API_KEY }},
    #       and IMAGE_NAME=citybike:${{ github.sha }} ./scripts/test-preprod.sh

  prod:
    needs: preprod
    runs-on: ubuntu-latest
    # TODO: same pattern with APP_ENV=production, PROD_API_KEY=${{
  secrets.PROD_API_KEY }},
    #       and ./scripts/test-prod.sh
```

Create a feature branch with a harmless change, push it to `ogn`, open a pull request to `main`, and watch the six jobs run in order. Do not claim a hosted check passed unless you saw it turn green on GitHub.

OR

Path B (Local Harness): If GitHub Actions is unavailable (or to test locally first), create scripts/pipeline.sh from scratch so that it takes an image tag as $1 (scripts/pipeline.sh citybike:v1), orchestrates the container lifecycle and environment variables for each of the five gates in sequence, stops immediately at the first failure, and leaves citybike-prod running on port 4173 after test-prod.sh passes:

```
# Create scripts/pipeline.sh, then run:
>>> chmod +x scripts/*.sh
>>> ./scripts/pipeline.sh citybike:v1
```

Expected**:** All five gates pass in sequence for `citybike:v1`.

Questions**:** A) Why must the gates run in order and stop at the first failure? B) What does each gate catch that the previous ones cannot? C) Why build the image once and promote it, instead of rebuilding for each environment? D) What is the configuration trap around `PROD_API_KEY`, and which gates catch it?

Mission 5 – Bad push vs. automated gate (35min)

Terminal A (Dev): add this "new feature" to `server.js`, at the top level right after the `API_KEY` line:

```
// v2: log the db host (needs DB_URL)
const dbHost = new URL(process.env.DB_URL).hostname;
console.log('db host:', dbHost);
```

Part 1 - No gate. Test it on your machine, where `DB_URL` is set in your shell. “It works on my machine”. Stop the local server, build `citybike:v2`, and deploy it straight to `prod`, skipping the pipeline:

```
>>> DB_URL=postgres://localhost:5432/app npm start
# Verify it starts cleanly, then press Ctrl+C to free port 4173

>>> docker build -t citybike:v2 .
>>> docker rm -f citybike-prod
>>> docker run -d --name citybike-prod -p 4173:4173 -e APP_ENV=production -e
... PROD_API_KEY=lab-secret citybike:v2
```

Terminal B (QA): assess the outage. Do not fix the code before you have diagnosed the failure and restored service.

```
# TODO: list all containers, including stopped ones (docker ps -a)
# TODO: call http://localhost:4173/health
# TODO: read the container logs (docker logs citybike-prod) and find the root cause
```

Decision (write it in your report): Rollback or fix forward, and why? Then roll `prod` back to the last known good image (`citybike:v1`) and verify it answers:

```
# TODO: replace citybike-prod with citybike:v1 (same port and environment variables)
and verify /health
```

Part 2 - With the gate. Terminal A puts the same `v2` change on a feature branch and sends it through the Mission 4 pipeline: (Use path chosen in prev mission)

- **Path A (GitHub Actions):** Push `feature/v2` to `ogn` and open a pull request to main.
- **Path B (Local Harness):** Run `./scripts/pipeline.sh citybike:v2` (ensure your `pipeline.sh` runs temporary gate containers `citybike`-gate on host port `4174` with `TARGET_URL=http://localhost:4174` so the live `citybike-prod` container on port `4173` is not killed unless `preprod` passes).

```
>>> git switch -c feature/v2
>>> git add server.js && git commit -m 'v2: log the db host'
>>> git push -u ogn feature/v2
```

Mission 6 –Deliverables (10min)

- Terminal output of the crash on `node:16` and the success on `node:20` (Mission 1).
- The size and time table for naive, cache fixed and multi-stage, the `BUILD ERROR` from the --omit=dev shortcut, and the output of ls node_modules failing in citybike:v1.
- The completed `sdlc.yml` (or `scripts/pipeline.sh`) utilizing the provided gate scripts, with the green run for v1
- Output of `docker ps -a`, `/health` and `docker logs` during the bad push, and the output of the rollback.
- Evidence that the pipeline blocked `v2` before prod.
- Git commit history and, for Path A, the pull request URL plus the Actions results.
- Answers to all the questions, plus: What should happen when a gate is red? Why practice the rollback before you need it?

Assessment rubric (preliminary; subject for change by professor)

Evidence	Points
Clean environment, crash reproduced and explained	10
Naive Dockerfile measured (time and size)	15
Cache fix and multi-stage, with proof that `devDependencies` are gone	20
The completed `sdlc.yml` (or `scripts/pipeline.sh`) utilizing the provided gate scripts, with the green run for v1	20
Bad push detected, rollback decided and applied to citybike:v1	15
v2 blocked by the pipeline before prod, with evidence	15
Evidence and clear explanation	5
TOTAL	100

Quick Troubleshooting

Problem	Try this
require is not defined in ES module scope	package.json has "type": "module" but the scripts use require: remove that line
BUILD ERROR: Necessary devDependencies missing	Expected with --omit=dev; otherwise run the build in the builder stage
Port 4173 is busy	Stop the previous npm start or run docker rm -f citybike-prod
Container name already in use	docker rm -f <name>
Container exits right away	docker logs <name> shows why
npm ci fails	Run npm install first and commit package-lock.json
Permission denied on a script	chmod +x scripts/*.sh
The workflow does not trigger	Check the on: block, the branch name and that Actions is enabled on the repo
First build is slow	The first node:20 pull is large, pull it in Mission 0

Reference material

Official documentation:

- https://docs.docker.com/build/building/multi-stage/
- https://docs.docker.com/build/cache/
- https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions